Guide Stratégique de Prompts QA Automation

Workflow piloté par Claude via MCP : Jira, Gherkin & Selenium

Claude MCP Jira Selenium Cucumber / BDD

Ce guide réunit les prompts utilisés pour automatiser le cycle de vie des tests avec **Claude**, connecté à **Jira** et **Selenium** grâce au **Model Context Protocol (MCP)**.

Après la configuration des deux serveurs MCP, un workflow simple couvre trois actions : **la création des scénarios**, **l'exécution en temps réel** et **la déclaration de bug** après l'exécution.

Vue d'ensemble

#	Prompt	MCP	Sortie
00	Connexion à Jira	Jira	Liste des projets
01	Initialisation & ticket Jira	Jira	Ticket QA en To Do (KAN-20)
02	Conception BDD (Gherkin)	-	Fichier login.feature
03	Exécution Selenium & synchronisation	Selenium + Jira	KAN-20 -> Done, rapport
04	Workflow amélioration & reporting	Selenium + Jira	KAN-21 -> In Progress
05	Gestion des anomalies	Jira	Bug KAN-22 lié à KAN-21

Le workflow en un coup d'oeil

De la spécification au rapport de bug, chaque étape correspond à un prompt.

Claude orchestre chaque étape via le Model Context Protocol (MCP) : Jira pour les tickets, Selenium pour le navigateur.

Prérequis

- Claude Desktop avec les serveurs MCP **Atlassian (Jira)** et **Selenium** configurés.
- Un projet Jira (dans la démo : **My Software Team**, clé **KAN**).
- Une spécification du module à tester, jointe dans la conversation (ex. *Login*).
- Application testée : **https://www.saucedemo.com/**

Variables à remplacer

Variable	Description
<EMAIL_TESTEUR>	Email Atlassian du testeur à qui assigner les tickets
My Software Team	Nom de votre projet Jira
KAN-20, KAN-21	Clés de vos tickets

Connexion à Jira

00 - Vérifier que le MCP Atlassian fonctionne avant de lancer le workflow :

```
Connecte-toi à Jira et donne-moi la liste des projets existants.
```

01 Initialisation & ticket Jira

Rôle : Expert QA & Jira | **MCP :** Atlassian (Jira)

Prompt

```
Tu es un expert QA et Jira.
Lis seulement le fichier « Login » fourni en contexte.
Crée un ticket Jira de type "TODO" pour un testeur QA dans le projet
"My Software Team".

Contexte :
- Testeur assigné : <EMAIL_TESTEUR>
- Module : Login
- Objectif : automatiser les tests du module login

Le ticket doit contenir :
- Titre clair
- Description détaillée
- Objectif du test
- Périmètre (login valide / invalide / lock user / performance user etc.)
- Critères d'acceptation
- Definition of Done
- Tags (QA, automation, login, regression)

Format attendu : prêt à copier dans Jira.
```

Résultat obtenu

- Claude lit la spécification et retrouve l'identifiant Jira du testeur.
- Ticket **KAN-20** créé en **To Do**, priorité Medium, labels *QA, automation, login, regression*.

02 Conception BDD (Gherkin)

Rôle : Expert QA Automation | **MCP :** aucun (génération)

Prompt

```
Tu es un expert QA Automation.
Lis le document de spécification du module Login.

Actions requises :
1. Identifie toutes les fonctionnalités du login.
2. Génère tous les scénarios possibles de test (positifs, négatifs,
   edge cases).
3. Transforme ces scénarios en format Cucumber (Gherkin).

Format attendu :
Feature: Login
Scenario: [Nom du scénario]
Inclure : Given / When / Then, cas utilisateur valide/invalide/bloqué,
mot de passe incorrect, champs vides et comportement UI attendu.
```

Exemple de scénario généré

```
@smoke @regression
Scenario: Connexion réussie avec l'utilisateur standard
  Given je suis sur la page de connexion "https://www.saucedemo.com/"
  When je saisis le username "standard_user"
  And je saisis le password "secret_sauce"
  And je clique sur le bouton "Login"
  Then je suis redirigé vers la page d'inventaire "/inventory.html"
  And le titre de la page est "Swag Labs"
```

Le fichier complet couvre les cas positifs, négatifs, edge cases, session, sécurité et performance (*features/login.feature* dans le dépôt GitHub).

03 Exécution & synchronisation

Rôle : Exécutant QA Automation | **MCP :** Selenium + Atlassian (Jira)

Prompt

```
Fais l'exécution avec Selenium de ce scénario :
Scenario: Connexion réussie avec l'utilisateur standard

Workflow post-exécution :
1. Change le statut du ticket "QA Login" sur Jira.
2. Ajoute un commentaire détaillant le résultat du test au ticket KAN-20.
```

Résultat obtenu

Indicateur	Valeur
Étapes réussies	8 / 8
Taux de réussite	100 %
Durée d'exécution	~3 secondes
Navigateur	Chrome (Selenium WebDriver)
Ticket KAN-20	To Do -> Done, rapport complet en commentaire

04 Workflows avancés

Deux prompts qui enchaînent tout le cycle et gèrent automatiquement les échecs.

4.1 Amélioration & reporting

```
Tu es un QA Automation Engineer expert en Jira, Cucumber et Selenium.
Objectif : lire uniquement le fichier « Amélioration » fourni et
automatiser tout le workflow QA autour de cette amélioration.

Étape 1 - Analyse du fichier d'amélioration.
Étape 2 - Création du ticket Jira TODO
          (Labels : QA, Automation, Login, Enhancement
           | Assigné à : <EMAIL_TESTEUR>).
Étape 3 - Génération d'un seul scénario Gherkin d'acceptation.
Étape 4 - Exécution du test Selenium.
Étape 5 - Mise à jour Jira :
          - Commentaire de reporting.
          - Si succès -> DONE.
          - Si échec  -> IN PROGRESS.
```

Résultat obtenu

Étape	Résultat
1 - Analyse	Ajout de l'utilisateur Claude123 / secret_sauce
2 - Jira	Ticket KAN-21 créé en To Do
3 - Gherkin	1 scénario @enhancement @acceptance @login
4 - Selenium	**FAILED** : Claude123 non reconnu
5 - Jira	KAN-21 -> In Progress, rapport posté

4.2 Gestion des anomalies (bug tracking)

```
Analyse le ticket d'amélioration après l'exécution des tests.
Si un bug est détecté :
- Créer un nouveau ticket BUG en statut TODO.
- Ajouter la description détaillée de l'anomalie.
- Lier techniquement le ticket BUG au ticket QA Enhancement concerné.
- Assigner le ticket à : <EMAIL_TESTEUR>
```

Résultat obtenu

- Ticket **KAN-22** : *[BUG] L'utilisateur Claude123 n'est pas reconnu par l'application*, priorité **High**.
- Rapport structuré : étapes de reproduction, comportement attendu / observé, données de l'assertion Selenium.
- Lien Jira **KAN-22 blocks KAN-21** + commentaire de traçabilité sur KAN-21.

Bonnes pratiques de prompting

Ce qui rend ces prompts fiables et reproductibles.

Technique	Exemple	Pourquoi
Rôle explicite	« Tu es un expert QA et Jira »	Cadre le niveau d'expertise et le vocabulaire
Contexte limité	« Lis seulement le fichier Login »	Évite le mélange de sources
Étapes numérotées	Étape 1 ... Étape 5	Workflow déterministe et traçable
Format imposé	Feature / Scenario / Given-When-Then	Sortie directement exploitable
Règles conditionnelles	Succès -> Done, échec -> In Progress	La décision est automatisée
Critères de fin	Definition of Done dans le ticket	Le résultat est vérifiable

Ressources

- Dépôt GitHub : **github.com/adama007/claude-mcp-qa-automation**
- Configuration des serveurs MCP : *docs/mcp-setup.md*
- Rapports générés : *reports/KAN-20_execution_report.md*, *reports/KAN-22_bug_report.md*

Guide généré pour l'optimisation des processus de test logiciel.